Project 3: Smooth Deconvolution of COVID-19 Data

Isaac Alan Yoho, Seán Connaire, Eoghan Smith

Overview & High-Level Outline

In this project, we fit a deconvolution model to Covid-19 death data from 2020. The goal is to recover the trajectory of the **infection curve** that led to the observed deaths.

Key Methodology:

- **Distribution:** We model deaths using a Poisson distribution.
- **Smoothing:** We use a penalized negative log-likelihood with a smoothing parameter λ to prevent overfitting.
- **Optimization:** The model is fitted using the "BFGS" method in `optim` with exact derivatives.
- **Selection:** The optimal λ is chosen by minimizing the Bayesian Information Criterion (BIC).
- **Uncertainty:** We use non-parametric bootstrapping to estimate a 95% confidence interval for the daily new infection rate.

We assume the interval from infection to death d follows a log-normal distribution:

$$\log(d) \sim N(3.151, 0.469^2)$$

Defining Core Functions

We define several functions to handle matrix initialization, likelihood calculation, and optimization.

1. Matrix Initialization

This function generates the model matrix X for deaths, X_{tilde} for infections, and the penalty matrix S.

```
init_model_matrices <- function(K, t, spline_range, min_window, max_window, edur, sdur) {
  inner_knots <- seq(min(spline_range), max(spline_range), length.out = K-2)
  int_length <- inner_knots[2] - inner_knots[1]

  knot_vals <- c(
    seq(min(spline_range) - (3*int_length), min(spline_range) - int_length, by = int_length),
    inner_knots,
    seq(max(spline_range) + int_length, max(spline_range) + (3*int_length), by = int_length)
  )

  X_tilde <- splineDesign(knots = knot_vals, x = spline_range)
  n <- length(t)
  d <- 1:max_window
  pd <- dlnorm(d, edur, sdur)
  pd <- pd / sum(pd)

  X <- matrix(0, nrow = n, ncol = K)
  for (i in 1:n) {
    window_width <- min(min_window - 1 + i, max_window)
```

```
    j <- 1:window_width
    rows <- min_window + i - j
    X[i, ] <- colSums(X_tilde[rows, , drop = FALSE] * pd[j])
  }

  S <- crossprod(diff(diag(K), diff = 2))
  return(list(X_tilde = X_tilde, X = X, S = S))
}
```

2. Likelihood and Gradients

Functions to calculate the Penalized Negative Log-Likelihood (PNLL) and its gradient.

```
pnll <- function(gamma, X, y, S, lambda = 5e-5, wb = 1) {
  beta <- exp(gamma)
  mu <- as.vector(X %*% beta)
  nll <- -sum(wb * (y * log(mu) - mu))
  P <- 0.5 * lambda * (beta %*% S %*% beta)
  as.numeric(nll + P)
}

gll <- function(gamma, X, y, S, lambda = 5e-5, wb = 1) {
  beta <- exp(gamma)
  mu <- as.vector(X %*% beta)
  v <- wb * ((y / mu) - 1)
  grad <- - beta * (t(X) %*% v) + lambda * (beta * (S %*% beta))
  as.vector(grad)
}

fd <- function(gamma, X, y, S, lambda = 5e-5, eps = 1e-7) {
  fd_vec <- numeric(length(gamma))
  pnll0 <- pnll(gamma, X, y, S, lambda)
  for (i in 1:length(gamma)) {
    gamma1 <- gamma
    gamma1[i] <- gamma1[i] + eps
    pnll1 <- pnll(gamma1, X, y, S, lambda)
    fd_vec[i] <- (pnll1 - pnll0) / eps
  }
  fd_vec
}
```

3. Model Selection (BIC) and Bootstrapping

```
BIC <- function(lambda, beta_hat, y, X, S) {
  n <- length(y)
  mu <- as.vector(X %*% beta_hat)
  W_v <- y / mu^2
  H0 <- t(X * W_v) %*% X
  H_lam <- H0 + lambda * S
  R <- chol(H_lam)
  H_lam_inv <- backsolve(R, forwardsolve(t(R), diag(ncol(H_lam))))
  EDF <- sum(H_lam_inv * t(H0))
  nll <- -sum(y * log(mu) - mu)
  2 * nll + log(n) * EDF
```

```r
}

minimize_BIC <- function(range = seq(-13, -7, length = 50), BIC, par, fn, gr, y, X, S) {
  lambda <- exp(range)
  BICs <- numeric(length(lambda))
  for (i in seq_along(lambda)) {
    fit_i <- optim(par = par, fn = fn, gr = gr, y = y, X = X, S = S, lambda = lambda[i], method = 'BFGS
    beta_hat <- exp(fit_i$par)
    BICs[i] <- BIC(lambda[i], beta_hat = beta_hat, y = y, X = X, S = S)
  }
  return(list(lambda = lambda[which.min(BICs)], BICs = cbind(lambda, BICs)))
}

best_fit <- function(par, fn, gr, y, X, S, lambda) {
  fit <- optim(par = par, fn = fn, gr = gr, y = y, X = X, S = S, lambda = lambda, method = 'BFGS')
  beta_hat <- exp(fit$par)
  mu_hat <- as.vector(X %*% beta_hat)
  # Note: Use global X_tilde_config inside if needed or pass as argument
  f_hat <- as.vector(X_tilde_config %*% beta_hat)
  return(list(beta_hat = beta_hat, mu_hat = mu_hat, f_hat = f_hat))
}

np_boot <- function(nb = 200, par, fn, gr, y, X, X_tilde, S, lambda) {
  n <- length(y)
  boot_f <- matrix(0, nrow = nrow(X_tilde), ncol = nb)
  for (i in 1:nb) {
    wb <- tabulate(sample(n, replace = TRUE), n)
    opt_boot <- optim(par = par, fn = fn, gr = gr, y = y, wb = wb, X = X, S = S, lambda = lambda, metho
    beta_hat_boot <- exp(opt_boot$par)
    boot_f[, i] <- X_tilde %*% beta_hat_boot
  }
  f_ci_lower <- apply(boot_f, 1, function(x) quantile(x, 0.025))
  f_ci_upper <- apply(boot_f, 1, function(x) quantile(x, 0.975))
  return(list(f_ci_lower = f_ci_lower, f_ci_upper = f_ci_upper))
}
```

Data Import and Configuration

```r
# Load your data here.
# Ensure "engcov.txt" is in the same directory as this Rmd.
data <- read.table("engcov.txt", header = TRUE)
t_config <- as.vector(data$julian)
y_config <- as.vector(data$nhs)

K_config <- 80
min_window_config <- 30
max_window_config <- 80
edur_config <- 3.151
sdur_config <- .469

spline_range_config <- seq(from = min(t_config) - min_window_config, to = max(t_config))
gamma_config <- rep(0, K_config)
```

```r
# Initialize Matrices
model_matrices <- init_model_matrices(K_config, t_config, spline_range_config,
                                      min_window_config, max_window_config,
                                      edur_config, sdur_config)
X_tilde_config <- model_matrices$X_tilde
X_config <- model_matrices$X
S_config <- model_matrices$S
```

Gradient Validation

We check the analytical gradient against a finite difference approximation to ensure accuracy.

```r
fd_out <- fd(gamma_config, X = X_config, y = y_config, S = S_config)
grad_out <- gll(gamma_config, X = X_config, y = y_config, S = S_config)
knitr::kable(head(cbind(FiniteDifference = fd_out, AnalyticalGradient = grad_out)))
```

FiniteDifference	AnalyticalGradient
-5.45549	-5.455492
-43.89356	-43.893557
-78.01665	-78.016645
-97.36966	-97.369653
-120.13500	-120.134994
-147.74263	-147.742619

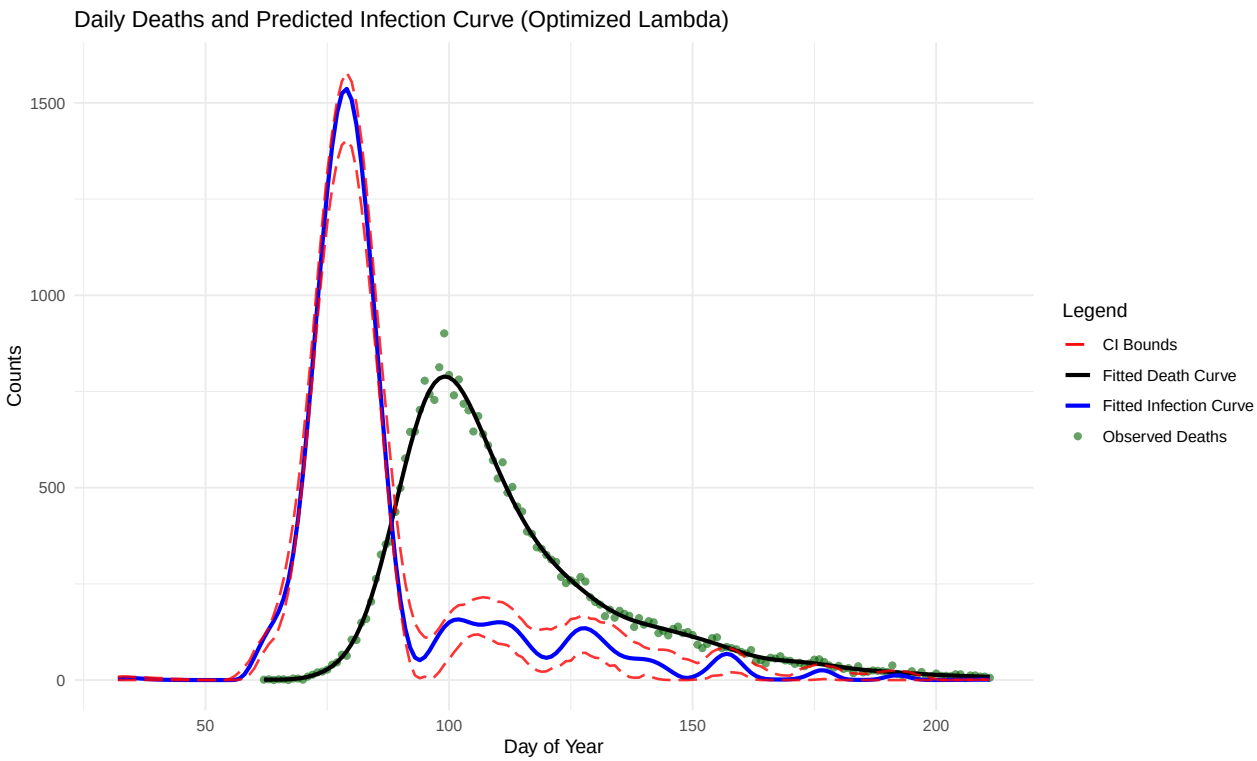
Model Fitting and BIC Optimization

We first fit an initial model, then optimize λ using the BIC criterion.

```r
# Initial arbitrary fit
M_0 <- best_fit(par = gamma_config, fn = pnll, gr = gll, y = y_config, X = X_config, S = S_config, lambd

# Optimize Lambda
search_range <- seq(-13, -7, length = 50)
min_BIC <- minimize_BIC(range = search_range, BIC = BIC, par = log(M_0$beta_hat),
                        fn = pnll, gr = gll, y = y_config, X = X_config, S = S_config)

lambda_hat <- min_BIC$lambda

# Optimized Model
M_1 <- best_fit(par = log(M_0$beta_hat), fn = pnll, gr = gll, y = y_config,
                X = X_config, S = S_config, lambda = lambda_hat)

# Bootstrapping for CI
bootstrap <- np_boot(nb = 200, par = log(M_1$beta_hat), fn = pnll, gr = gll,
                     y = y_config, X = X_config, X_tilde = X_tilde_config,
                     S = S_config, lambda = lambda_hat)
```

Results and Visualizations

Final Optimized Model with Confidence Intervals

This plot shows the observed death counts, our fitted death curve ($\hat{\mu}$), and the deconvolved infection trajectory ($\hat{f}$) with its 95% bootstrap confidence interval.

```r
ggplot() +
  geom_point(aes(x = t_config, y = y_config, color = "Observed Deaths"), size = 1.5, alpha = 0.6) +
  geom_line(aes(x = t_config, y = M_1$mu_hat, color = 'Fitted Death Curve'), linewidth = 1.1) +
  geom_line(aes(x = spline_range_config, y = M_1$f_hat, color = 'Fitted Infection Curve'), linewidth = 
  geom_line(aes(x = spline_range_config, y = bootstrap$f_ci_lower, color = 'CI Bounds'),
            linetype = "longdash", linewidth = .7, alpha = 0.8) +
  geom_line(aes(x = spline_range_config, y = bootstrap$f_ci_upper, color = 'CI Bounds'),
            linetype = "longdash", linewidth = .7, alpha = 0.8) +
  scale_color_manual(
    name = "Legend",
    values = c("Observed Deaths" = "darkgreen", "Fitted Death Curve" = "black",
               "Fitted Infection Curve" = "blue", "CI Bounds" = "red")
  ) +
  labs(x = "Day of Year", y = "Counts",
       title = "Daily Deaths and Predicted Infection Curve (Optimized Lambda)") +
  theme_minimal()
```